

A Study of Q-Learning and Deep Q-Networks on the Taxi-v3 Problem

Aniruddha Datta, Sahasrangshu Roy, Niladri Banerjee

Department of Computer Science
Ramakrishna Mission Vivekananda Educational and Research Institute

Abstract

Reinforcement Learning (RL) is a machine learning paradigm in which an agent learns optimal behavior through interaction with an environment by maximizing cumulative reward. This project investigates the application of reinforcement learning techniques to solve the Taxi-v3 environment, a classical discrete Markov Decision Process. Two approaches are implemented and analyzed: Tabular Q-Learning and Deep Q-Networks (DQN). The objective is to train an autonomous taxi agent capable of navigating a grid world, correctly picking up passengers, and delivering them to designated destinations while minimizing penalties and travel time. Q-Learning is used to establish a baseline solution through a state-action value table, while DQN employs a neural network to approximate the action-value function for improved scalability. Experimental results demonstrate successful learning in both approaches, with DQN showing stable convergence and the ability to generalize across states. The study highlights the strengths and limitations of tabular and deep reinforcement learning methods and illustrates the effectiveness of deep reinforcement learning in sequential decision-making problems.

Overview

Reinforcement Learning (RL) is a branch of machine learning in which an agent learns to make decisions by interacting with an environment and receiving feedback in the form of rewards. This project focuses on solving the Taxi-v3 problem, a classical discrete-state RL environment, using two different approaches: Tabular Q-Learning and Deep Q-Networks (DQN).

The objective is to train an autonomous taxi agent to efficiently pick up and drop off passengers at designated locations while minimizing penalties and maximizing cumulative rewards. The project demonstrates the limitations of tabular methods and how deep learning can be used to overcome them.

Introduction

Traditional supervised learning techniques require labeled data, whereas reinforcement learning enables an agent to

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

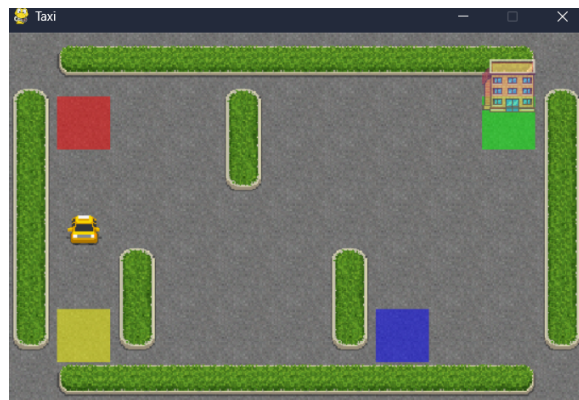


Figure 1: Gymnasium Taxi-v3 environment.

learn optimal behavior through trial and error. The Taxi-v3 environment is widely used as a benchmark problem to study reinforcement learning algorithms due to its simplicity and well-defined reward structure.

In this project, we first apply Q-Learning, a model-free RL algorithm, to solve the problem using a Q-table. Subsequently, we extend the solution using a Deep Q-Network, which replaces the Q-table with a neural network to approximate the action-value function. The performance of both methods is evaluated and compared.

Problem Statement, Formulation and Representation

Environment Description

The environment consists of a 5×5 grid with walls that restrict movement. Four special locations (R, G, Y, B) represent passenger pickup points and destinations.

State and Action Space

The state space consists of 500 discrete states, each encoding:

- Taxi position (row, column)
- Passenger location (including in-taxi)
- Destination location

The action space contains six discrete actions:

- 0–3: South, North, East, West
- 4: Pickup passenger
- 5: Drop off passenger

Reward Structure

- +20 for successful passenger drop-off
- 10 for illegal pickup or drop-off
- 1 per time step

Environment

The project uses the `Gymnasium` library to simulate the Taxi-v3 environment. Episodes begin from random valid states and terminate upon successful passenger delivery or episode truncation.

Problem Statement

The goal is to design an intelligent taxi agent that can:

- Navigate a grid-based environment
- Pick up a passenger from a given location
- Drop the passenger at the correct destination
- Minimize penalties and total time steps

Mathematical Formulation

The Taxi-v3 problem is formulated as a Markov Decision Process (MDP):

- States (S): 500 discrete states
- Actions (A): 6 possible actions
- Transition Model (P): Deterministic
- Reward Function (R):
 - +20 for successful drop-off
 - -10 for illegal pickup/drop-off
 - -1 per time step
- Discount Factor (γ): $0 < \gamma \leq 1$

The objective is to learn an optimal policy π^* that maximizes expected cumulative reward.

Hyperparameters

- Learning rate (α): 0.4
- Discount factor (γ): 0.6
- Initial exploration rate (ϵ): 0.7
- Minimum exploration rate: 0.01
- Episodes: 10,000

Algorithms Used

This project employs two reinforcement learning algorithms to solve the Taxi-v3 environment: **Tabular Q-Learning** and **Deep Q-Networks (DQN)**. Both algorithms aim to learn an optimal policy that maximizes cumulative reward, but they differ in how the action-value function is represented and learned.

Q-Learning

Q-Learning is a model-free, value-based reinforcement learning algorithm that learns the optimal action-value function $Q(s, a)$, which represents the expected cumulative reward obtained by taking action a in state s and following the optimal policy thereafter.

The Q-value update rule is given by:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

where s is the current state, a is the chosen action, r is the received reward, s' is the next state, α is the learning rate, and γ is the discount factor.

The learning process follows these steps:

- Initialize the Q-table with zeros.
- At each step, select an action using an ϵ -greedy policy.
- Execute the action and observe the reward and next state.
- Update the Q-value using the Bellman equation.
- Repeat the process over multiple episodes until convergence.

Q-Learning performs effectively in the Taxi-v3 environment due to its small and discrete state space. However, it becomes impractical for larger environments because the Q-table grows rapidly with the number of states.

Deep Q-Network (DQN)

Deep Q-Network (DQN) extends Q-Learning by replacing the Q-table with a neural network that approximates the action-value function:

$$Q(s, a) \approx Q_{\theta}(s, a)$$

where θ represents the network parameters.

In this project, states are represented using one-hot encoding and passed through a fully connected neural network with two hidden layers. The output layer produces Q-values corresponding to each possible action.

To improve training stability, DQN incorporates the following techniques:

- **Experience Replay:** Transitions $(s, a, r, s', done)$ are stored in a replay buffer and randomly sampled during training to break correlations between consecutive experiences.
- **Target Network:** A separate target network is used to compute stable target Q-values and is periodically updated from the policy network.
- **ϵ -Greedy Exploration:** Balances exploration and exploitation during learning.

The target value for training the network is computed as:

$$y = r + \gamma \max_{a'} Q_{target}(s', a')$$

and the loss is minimized using mean squared error between predicted and target Q-values.

DQN enables learning in environments with larger or more complex state spaces and demonstrates stable convergence in the Taxi-v3 problem, albeit with higher computational cost compared to tabular Q-Learning.

Comparison of Algorithms

Feature	Q-Learning	DQN
State Representation	Q-Table	Neural Network
Scalability	Low	High
Memory Requirement	High for large states	Fixed
Training Stability	High	Moderate
Computational Cost	Low	High

Overall, Q-Learning serves as a strong baseline for small environments, while DQN illustrates the power of deep reinforcement learning in handling larger and more complex problems.

Implementation

Tools and Libraries

- Python
- Gymnasium
- Numpy
- Pytorch

Natural Network Architecture

- Input layer: 500 neurons (one-hot state representation)
- Two hidden layers with 128 neurons each
- Numpy
- Output layer: 6 neurons (Q-values for each action)

Training Process

- Initialize environment and parameters
- Select actions using -greedy strategy
- Store experiences in replay buffer
- Sample mini-batches for training
- Update policy network using gradient descent
- Periodically update target network

Results

Q Learning Results

- Converges after several thousand episodes
- Achieves consistent positive rewards
- Performs well due to small state space

DQN Results

- Learns stable policy over time
- TReward curve shows gradual improvement
- Handles state representation more flexibly

The reward curves indicate successful learning, with the agent eventually completing tasks efficiently with minimal penalties.

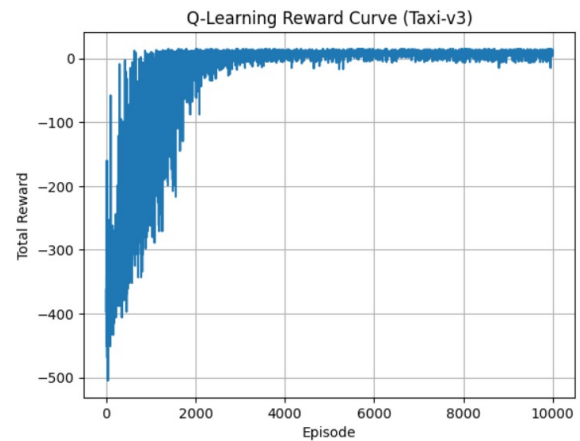


Figure 2: Training reward curve for the Q-Learning agent on the Taxi-v3 environment.

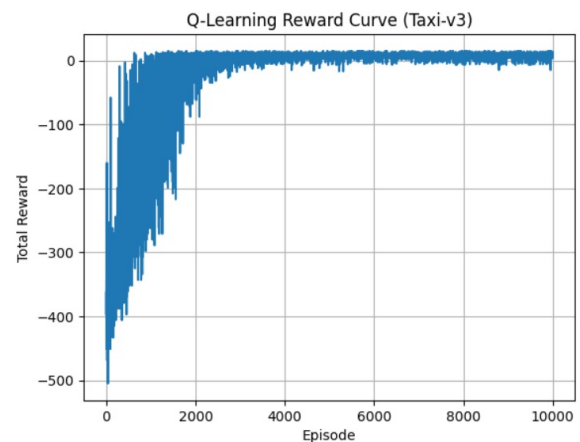


Figure 3: Training reward curve for the DQN agent on the Taxi-v3 environment.

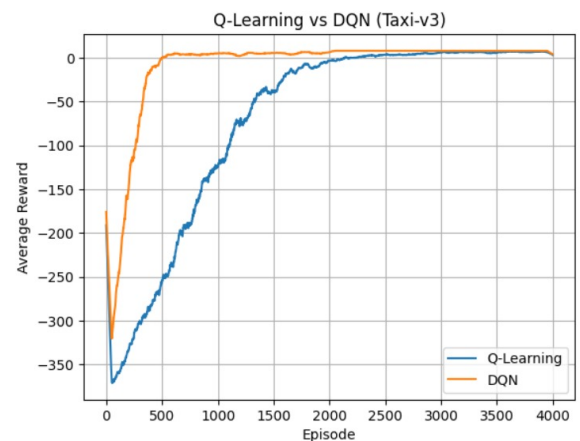


Figure 4: Comparison Curve: Q v/s DQN reward.

Discussion

While Q-Learning performs effectively in Taxi-v3 due to the limited state space, it becomes impractical for larger environments because the Q-table grows exponentially. DQN addresses this limitation by using a neural network, making it scalable to more complex problems.

However, DQN introduces challenges such as instability and higher computational cost. Techniques like experience replay and target networks are crucial for stable learning.

Conclusion

This project successfully demonstrates the application of reinforcement learning to solve the Taxi-v3 environment using both tabular and deep learning approaches. Q-Learning provides a simple and interpretable solution, while DQN offers scalability and flexibility for larger state spaces.

The results highlight the strengths and limitations of each method and emphasize the importance of deep reinforcement learning in solving real-world sequential decision-making problems.

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.
- [2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.
- [3] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.
- [4] Farama Foundation, "Gymnasium Documentation," <https://gymnasium.farama.org/>
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
- [6] PyTorch Team, "PyTorch Documentation," <https://pytorch.org/docs/>